Subject: ASD&D	**Semester:** V
Class: TE	**Batch:** T11
Name: Ananya Amesur	**Roll No:** 2401005

Experiment 5

Aim: To automate application deployment using Github/Jenkins.

Lab Outcome: Mapped LO3

LO3: Build and manage automated CI/CD pipelines for application development, testing, packaging and deployment.

Theory:

Continuous Integration (CI) and Continuous Deployment (CD) form a fundamental DevOps practice aimed at bridging the gap between development and operational activities.

- **Continuous Integration (CI)**: A software development practice where developers regularly merge code changes into a central repository, followed by automated builds and automated tests. This helps catch bugs early, increases software quality, and reduces validation overhead.
- **Continuous Deployment (CD)**: The automated process where verified changes in the codebase are automatically packaged, configured, and deployed directly to production or staging runtime environments without manual intervention.
- **Jenkins & Declarative Pipelines**: Jenkins is an open-source automation server enabling automated build, test, and deployment workflows. A Declarative Pipeline uses structured Groovy-based domain-specific language (DSL) to define stages, steps, and build environments as code (Jenkinsfile), making infrastructure version-controlled and reproducible.

Step 1: Jenkins Server Initialization and Access Setup

- Jenkins runs as a background Windows service requiring a compatible Java Development Kit (JDK 21) runtime environment to execute its core processes.

- Accessing the web console at http://localhost:8080 allows the setup wizard to be completed, during which the first administrator user is created to manage permissions, job configurations and server tasks.

Step 2: Baseline Pipeline Validation (Inline Scripting)

- A new project item named GitHub-Jenkins-Demo is created using the native Pipeline job type.
- Before linking to external version control, an inline Declarative Pipeline script is added directly to the job configuration to test the server's execution engine.
- The script utilizes the agent any directive along with three core lifecycle stages: Build, Test, and Deploy, each executing standard echo statements.
- Running Build #1 causes Jenkins to assign a local node executor in the workspace directory, process each stage sequentially, and display the console logs showing that the pipeline completed with SUCCESS.

Step 3: Source Code Management (SCM) & Pipeline-as-Code Implementation

- A central GitHub repository named jenkins-demo-app is established under user profile to maintain the application codebase and its automation configuration.
- An index.html file is added as the frontend application artifact containing an initial header message.
- A version-controlled Jenkinsfile is defined in the root directory, following Declarative Pipeline syntax.
- In the Deploy stage of the Jenkinsfile, automated batch instructions (bat) are scripted to verify the destination directory C:\Deployments\DemoApp and copy index.html from the workspace to the target folder.

Step 4: Connecting Jenkins to GitHub and Initial Deployment

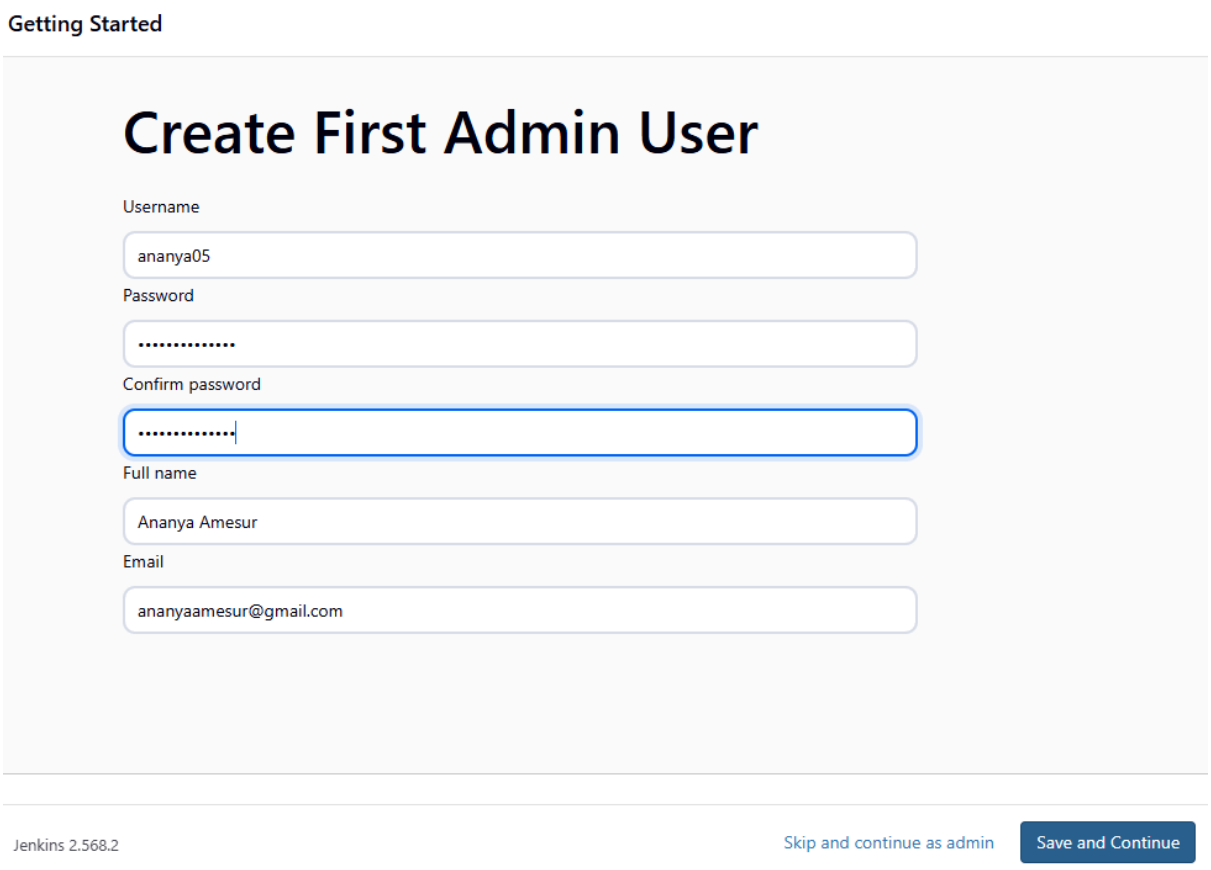
- The Jenkins project configuration is transitioned from an inline script to Pipeline script from SCM, specifying Git as the source provider.
- Jenkins is supplied with the repository HTTPS URL, the branch specifier is configured to */main, and the script path is set to target the repository's Jenkinsfile.
- Executing Build #2 prompts Jenkins to clone the remote repository, check out the latest commit, interpret the Jenkinsfile, and run the deployment batch script.

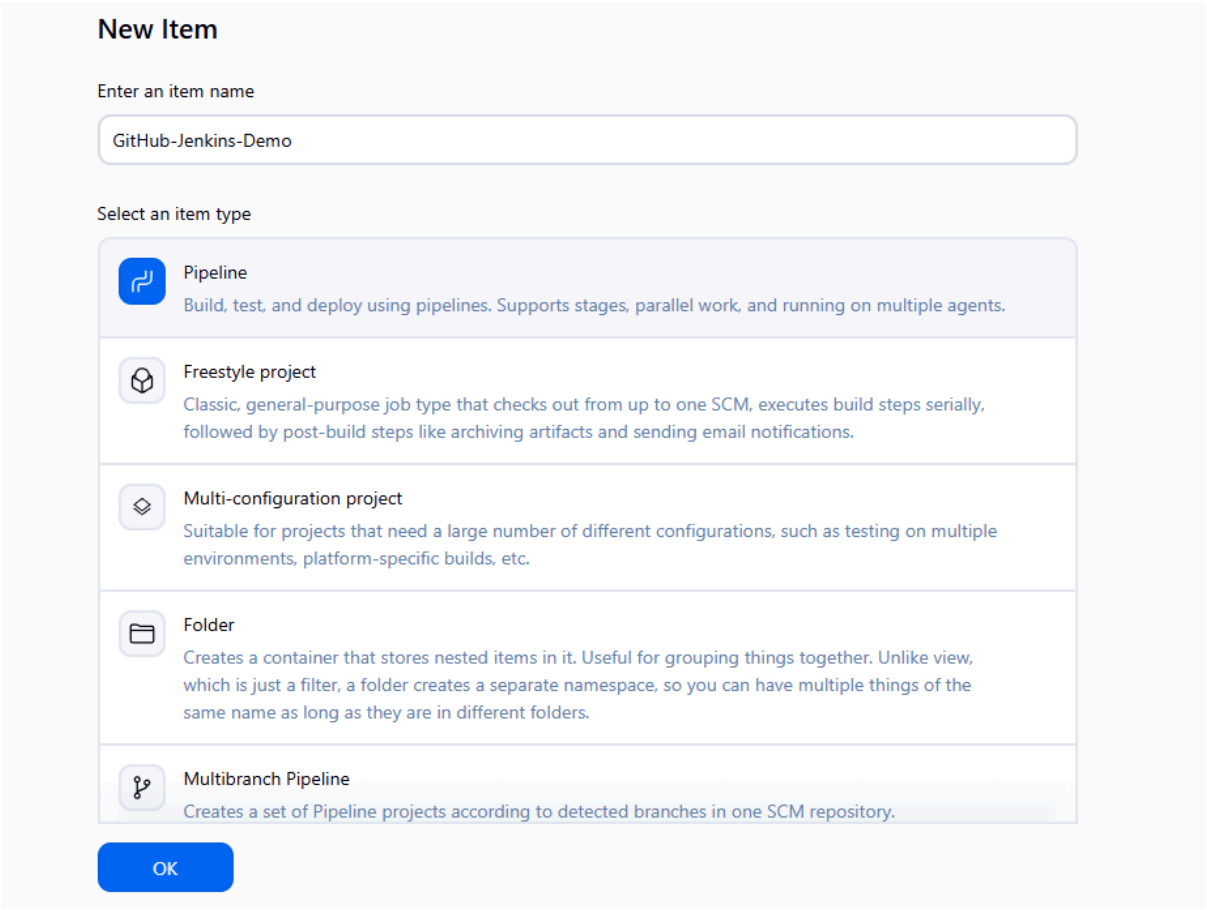
- Opening C:/Deployments/DemoApp/index.html in a web browser verifies that the initial frontend file was deployed successfully.

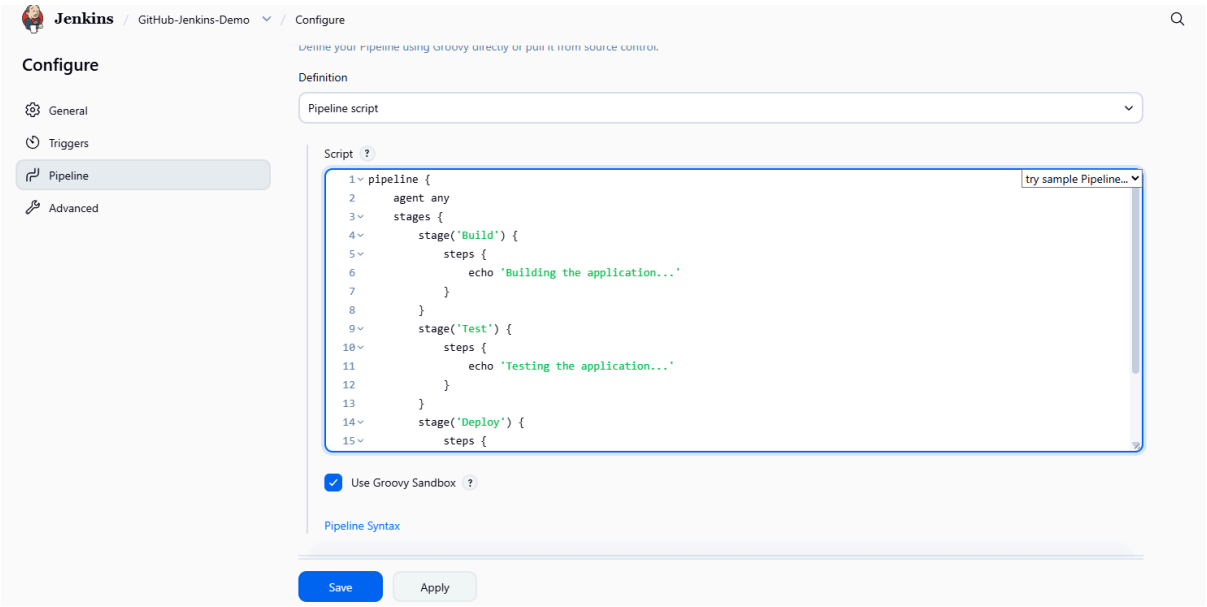
Step 5: Continuous Integration & Continuous Deployment (CI/CD) Verification

- To simulate real-world continuous deployment, a code change is made directly in index.html on GitHub by adding custom CSS styling and modifying the text to Application Updated & Deployed via CI/CD Pipeline! Version 2.0 is live..
- Upon running Build #3, Jenkins pulls the updated commit hash, executes the stages, and overwrites the existing index.html in C:\Deployments\DemoApp with the new version.
- Refreshing the browser page confirms that the new blue-themed card UI and Version 2.0 text are rendered, validating the full automated delivery pipeline from source control to local target deployment.

Execution:




New Item
Enter an item name
GitHub-Jenkins-Demo
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.
OK


Jenkins
GitHub-Jenkins-Demo
Configure
Configure
General
Triggers
Pipeline
Advanced
Definition
Pipeline script
Script
try sample Pipeline...
pipeline {
agent any
stages {
stage('Build') {
steps {
echo 'Building the application...'
}
}
stage('Test') {
steps {
echo 'Testing the application...'
}
}
stage('Deploy') {
steps {
Use Groovy Sandbox
Pipeline Syntax
Save
Apply

GitHub-Jenkins-Demo / #1

ut

'1'

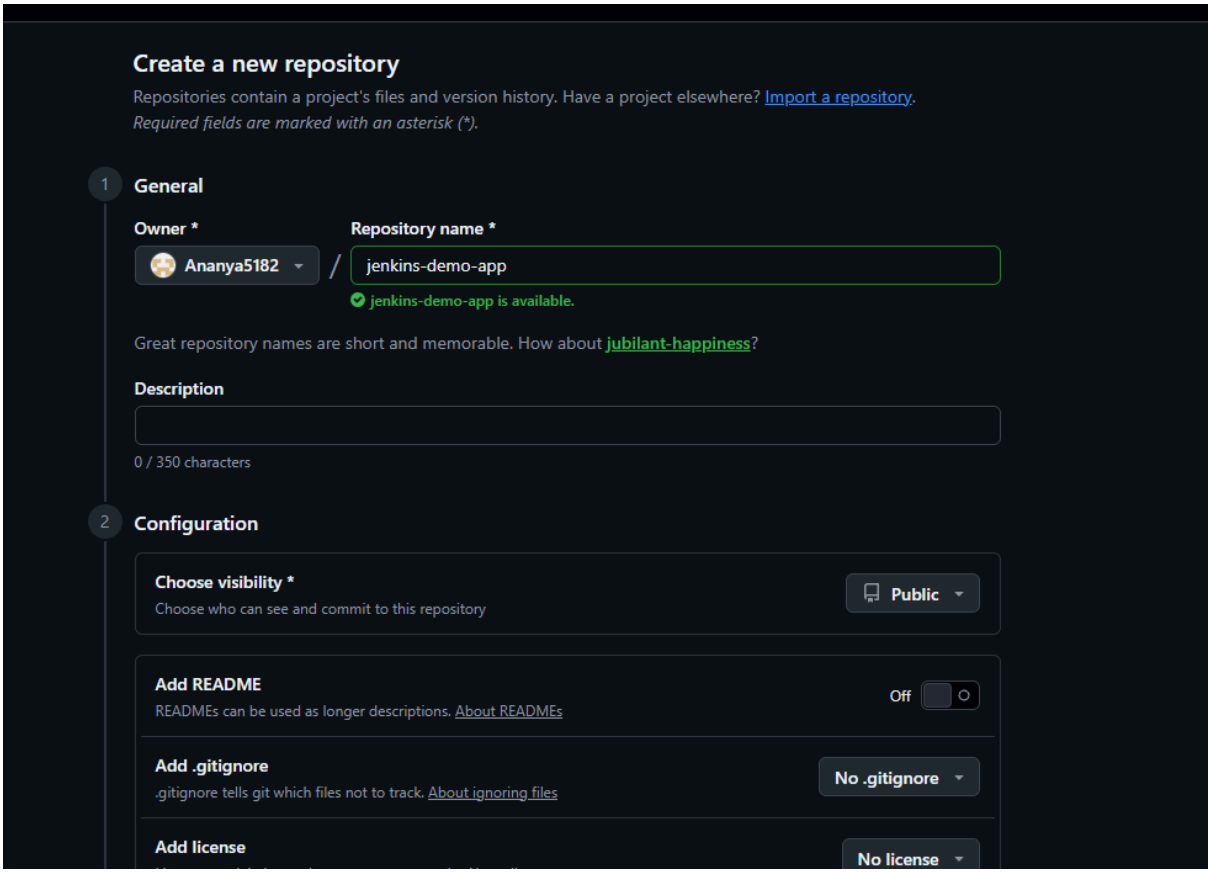
iew

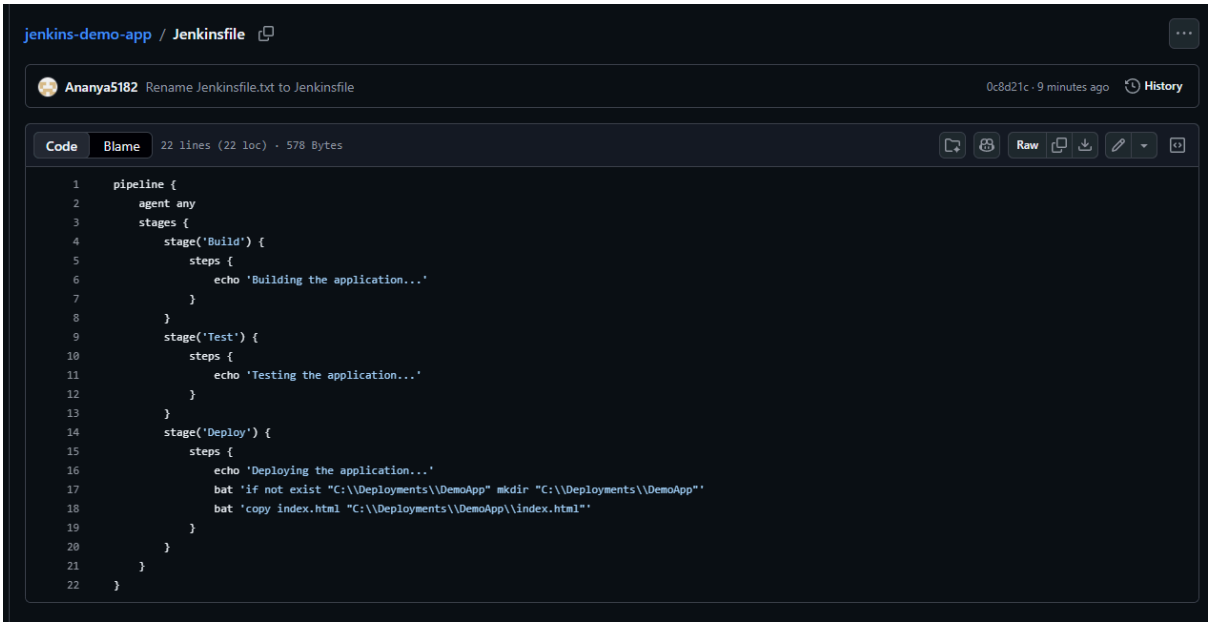
rmation

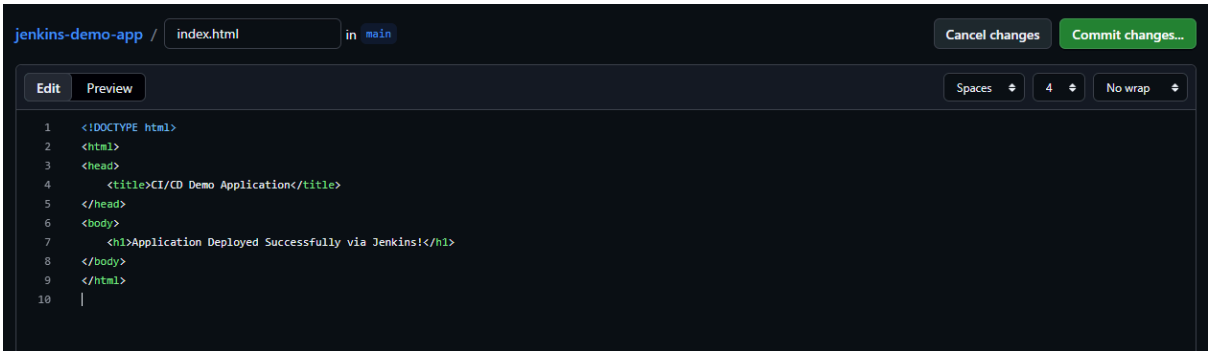
tage

Console

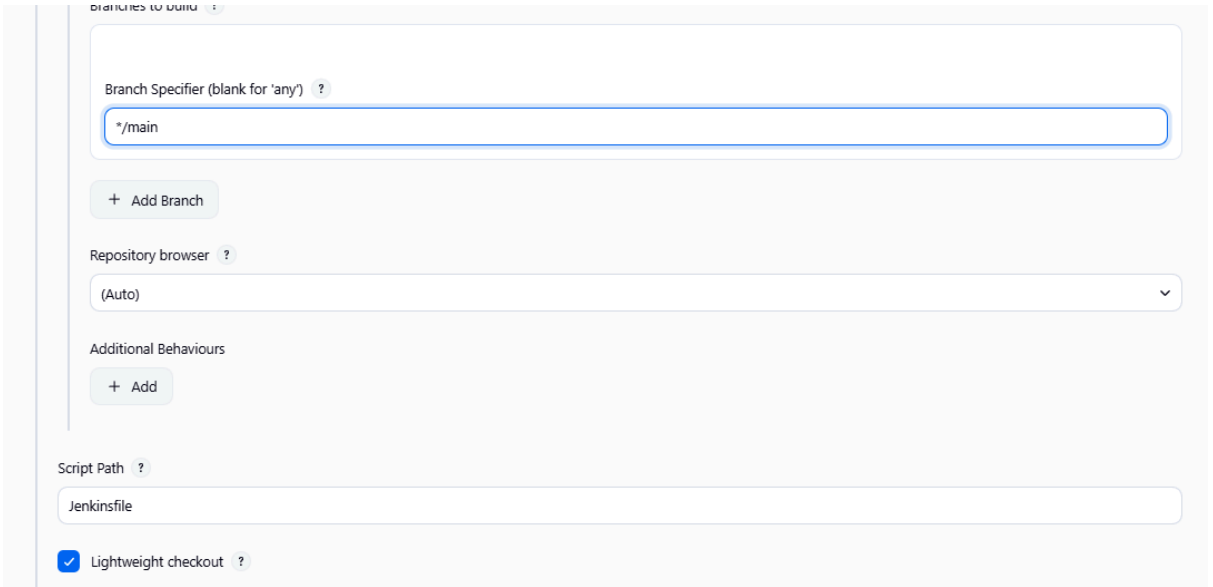
```
Started by user Ananya Amesur
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\GitHub-Jenkins-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

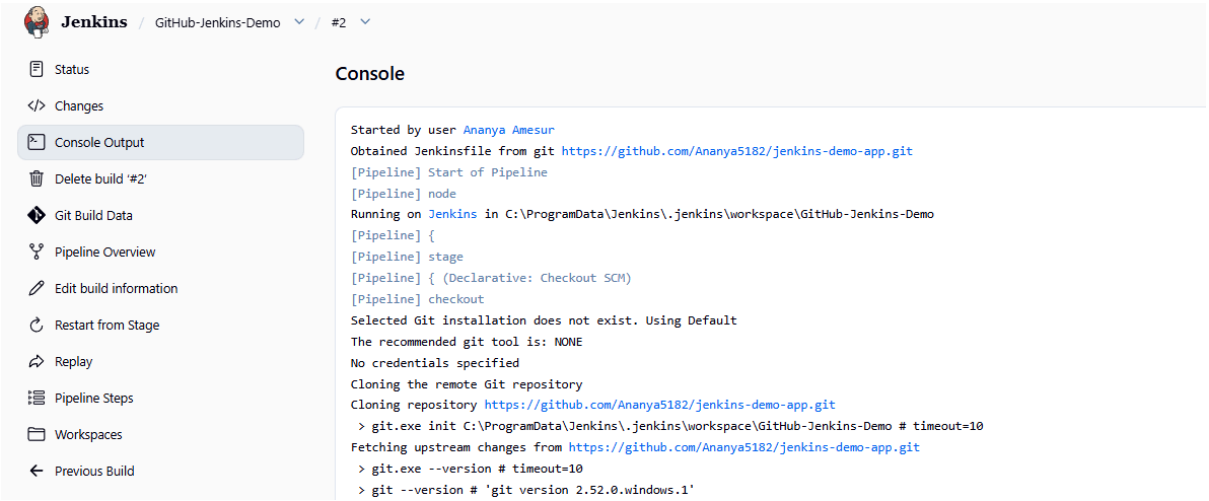

Create a new repository
Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.
Required fields are marked with an asterisk (*).
1
General
Owner *
Repository name *
Ananya5182
/
jenkins-demo-app
jenkins-demo-app is available.
Great repository names are short and memorable. How about jubilant-happiness?
Description
0 / 350 characters
2
Configuration
Choose visibility *
Choose who can see and commit to this repository
Public
Add README
READMEs can be used as longer descriptions. About READMEs
Off
Add .gitignore
.gitignore tells git which files not to track. About ignoring files
No .gitignore
Add license
No license


jenkins-demo-app / Jenkinsfile
Ananya5182 Rename Jenkinsfile.txt to Jenkinsfile
0c8d21c · 9 minutes ago
History
Code
Blame
22 lines (22 loc) · 578 Bytes
Raw
pipeline {
agent any
stages {
stage('Build') {
steps {
echo 'Building the application...'
}
}
stage('Test') {
steps {
echo 'Testing the application...'
}
}
stage('Deploy') {
steps {
echo 'Deploying the application...'
bat 'if not exist "C:\\Deployments\\DemoApp" mkdir "C:\\Deployments\\DemoApp"'
bat 'copy index.html "C:\\Deployments\\DemoApp\\index.html"'
}
}
}
}


jenkins-demo-app / index.html in main
Cancel changes
Commit changes...
Edit
Preview
Spaces
4
No wrap
<!DOCTYPE html>
<html>
<head>
<title>CI/CD Demo Application</title>
</head>
<body>
<h1>Application Deployed Successfully via Jenkins!</h1>
</body>
</html>


Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script from SCM
SCM
Git
Repositories
Repository URL
https://github.com/Ananya5182/jenkins-demo-app.git


Branch Specifier (blank for 'any')
*/main
Add Branch
Repository browser
(Auto)
Additional Behaviours
Add
Script Path
Jenkinsfile
Lightweight checkout




```
 > git.exe --version # timeout=10
 > git --version # 'git version 2.52.0.windows.1'
 > git.exe fetch --tags --force --progress -- https://github.com/Ananya5182/jenkins-demo-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git.exe config remote.origin.url https://github.com/Ananya5182/jenkins-demo-app.git # timeout=10
 > git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
 > git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision 0c8d21cde227e66276fd9b2df19fc1815a852684 (refs/remotes/origin/main)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f 0c8d21cde227e66276fd9b2df19fc1815a852684 # timeout=10
Commit message: "Rename Jenkinsfile.txt to Jenkinsfile"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building the application...
```

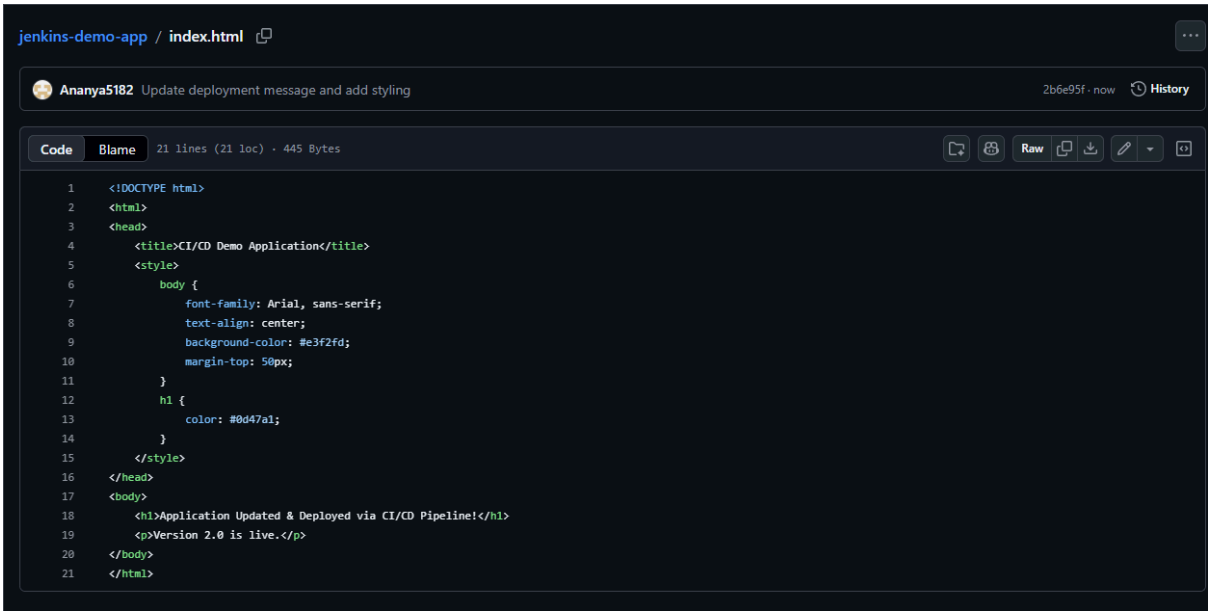
```
Building the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying the application...
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\GitHub-Jenkins-Demo>if not exist "C:\Deployments\DemoApp" mkdir "C:\Deployments\DemoApp"
[Pipeline] bat

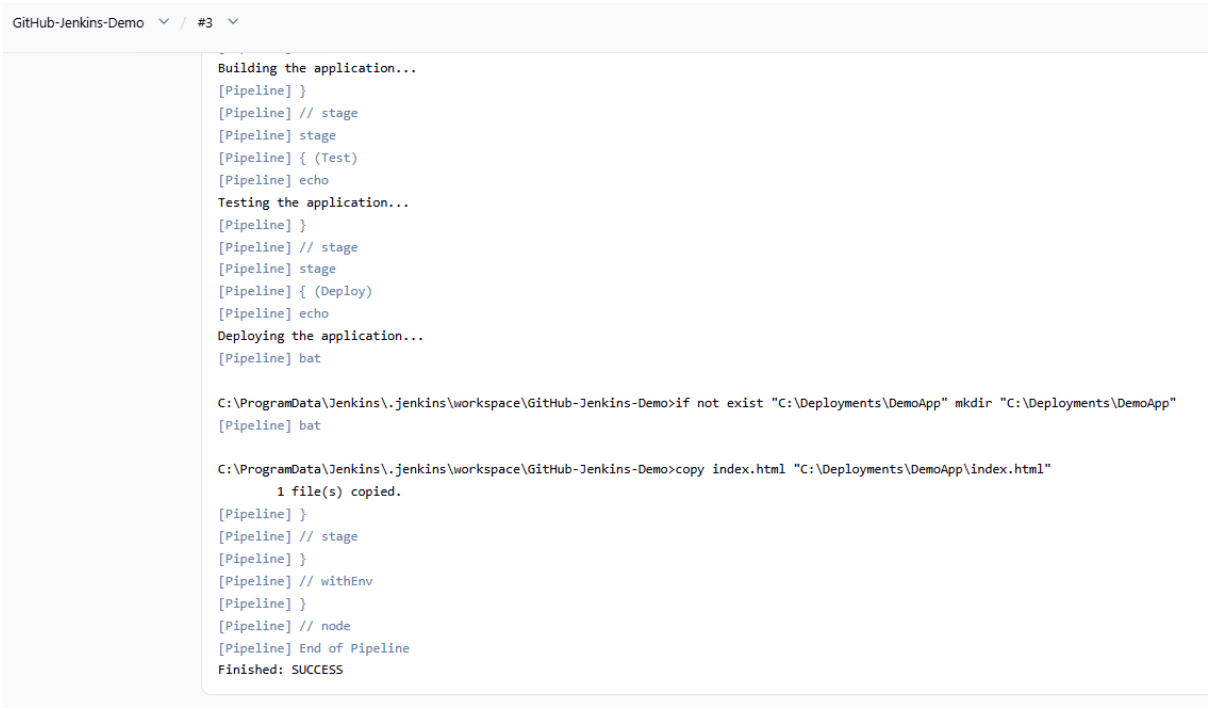
C:\ProgramData\Jenkins\.jenkins\workspace\GitHub-Jenkins-Demo>copy index.html "C:\Deployments\DemoApp\index.html"
        1 file(s) copied.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

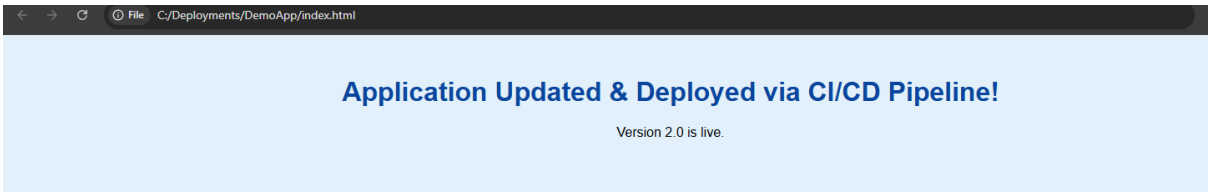


Application Deployed Successfully via Jenkins!




```html
<!DOCTYPE html>
<html>
<head>
    <title>CI/CD Demo Application</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            background-color: #e3f2fd;
            margin-top: 50px;
        }
        h1 {
            color: #0d47a1;
        }
    </style>
</head>
<body>
    <h1>Application Updated & Deployed via CI/CD Pipeline!</h1>
    <p>Version 2.0 is live.</p>
</body>
</html>
```





Conclusion:

Building and running an automated CI/CD pipeline by integrating Jenkins with GitHub bridges the gap between writing code and shipping it. Through Declarative Pipeline syntax, source code changes are automatically fetched, validated, and deployed to the target environment without manual effort. The primary takeaway is the practical value of Pipeline as Code via a Jenkinsfile, which keeps deployment logic version-controlled, traceable, and reproducible alongside the application. Ultimately, seeing a repository commit instantly reflected in the live setup proves how automated workflows eliminate human error, shorten release cycles, and ensure consistent software delivery.